

Working Plan (Student-Focused)

Traffic Control Optimization Through Video-Based Machine Learning

Lane-Free Online Intention Prior + Adaptive Signal Control

Project Leads: Zhaoxian Zhou/Ankit Devkota

February 12, 2026

Project Summary (What We Are Building)

We will build a real-time, camera-only adaptive traffic signal controller for a **single intersection with multiple approaches**. The system learns vehicle turning intentions *without lane modeling* by observing trajectories and self-labeling them using exit zones. It starts in **fixed-time mode** to collect data, then gradually transitions to **adaptive control** once its learned intention model is confident. After switching, the system **continues learning online** and keeps updating the intention model to adapt to changing traffic patterns.

System Workflow (End-to-End)

The system becomes:

1. Run fixed-time for data collection.
2. Track vehicles in the video stream.
3. Use exit zones to generate ground-truth movement labels (Left / Through / Right).
4. Build $P(m | c)$ online (a spatial intention prior) where c is a spatial cell.
5. Estimate real-time movement demand and traffic state.
6. Switch to adaptive control when confidence is sufficient.
7. Continue learning after switching; keep updating $P(m | c)$ online.

1 Key Design Choice: Lane-Free Spatial Intention Prior

1.1 Idea

Instead of detecting lane boundaries or lane groups, we discretize the observable roadway into spatial cells and learn:

$$P(m | c), \quad m \in \{L, T, R\},$$

meaning: “If a vehicle is currently in cell c , what is the probability it will eventually turn Left/Through/Right?”

1.2 What Is a Cell?

A “cell” can be:

- **Image grid cell** (e.g., 10×10 pixels), easiest to implement; or
- **Ground-plane cell** (recommended when possible) after homography mapping, e.g., 0.5×0.5 m or 1×1 m.

We will start with image-grid cells for speed, and optionally upgrade to ground-plane cells if calibration is stable.

1.3 Exit Zones (Self-Supervised Labels)

We manually define **exit zones** as polygons for each movement destination. When a tracked vehicle enters an exit zone, we assign its final label $m_k \in \{L, T, R\}$. This provides “free” training labels without manual per-frame intention labeling.

1.4 Online Update Rule (Continue Learning Forever)

We store per-cell counts (or decayed counts):

$$N_c, \quad N_{c,L}, N_{c,T}, N_{c,R}.$$

Batch (Count-Based) Update (Simple) After a vehicle completes and we know its final movement m_k , we update all cells visited by that vehicle within a chosen learning region (see below):

$$N_c \leftarrow N_c + 1, \quad N_{c,m_k} \leftarrow N_{c,m_k} + 1.$$

Then estimate:

$$P(m \mid c) = \frac{N_{c,m} + \alpha}{N_c + 3\alpha},$$

where $\alpha \in [0.5, 2]$ is a smoothing prior.

Online (EMA/Decay) Update (Recommended) To adapt to time-of-day changes, we use exponential decay:

$$N_{c,m} \leftarrow (1 - \beta)N_{c,m} + \beta \cdot \mathbf{1}[m = m_k], \quad N_c \leftarrow (1 - \beta)N_c + \beta,$$

where β is small (e.g., 0.001–0.01 depending on how quickly we want adaptation).

1.5 Learning Region (Avoid Ambiguous Zones)

To reduce ambiguity (cells where different trajectories overlap), we restrict updates to a **learning region**:

- include upstream approach areas (where vehicles “commit” to their movement),
- exclude the intersection center (where paths overlap),
- exclude far upstream regions if data is too noisy.

2 Traffic State Estimation (No Lane Assumptions)

Even without lanes, we still need a minimal traffic state for control. We define **approach ROIs** (polygons) for each incoming approach.

2.1 Queue Length per Approach

A vehicle is considered queued if its speed is below a threshold:

$$v < v_{\text{th}} \quad (\text{e.g., } v_{\text{th}} = 1 \text{ m/s or pixel-equivalent}).$$

Queue estimate for approach a :

$$q_a(t) = \#\{k : k \in \text{ROI}_a \wedge v_k(t) < v_{\text{th}}\}.$$

2.2 Arrival Rate per Approach

Define an upstream “entry line” (or narrow zone) per approach; count track crossings:

$$\lambda_a(t) = \frac{\#(\text{tracks crossing entry line in window})}{\Delta T}.$$

2.3 Real-Time Movement Demand (Turning Proportions)

For each tracked vehicle k currently in approach ROI a , find its current cell $c(k, t)$ and look up $P(m \mid c)$. Aggregate to estimate turning proportions:

$$\hat{\pi}_{a,m}(t) = \frac{1}{|\mathcal{V}_a(t)|} \sum_{k \in \mathcal{V}_a(t)} P(m \mid c(k, t)).$$

Estimated movement demand in a short horizon window can be approximated by:

$$\hat{d}_{a,m}(t) \propto q_a(t) \cdot \hat{\pi}_{a,m}(t).$$

3 Control Strategy: Safe Transition from Fixed-Time to Adaptive

3.1 Phase 0: Fixed-Time (Data Collection Mode)

Initially, the intersection runs a fixed-time plan (existing controller). Our system runs in **shadow mode**:

- detects and tracks vehicles,
- self-labels movements via exit zones,
- updates $P(m | c)$ online,
- computes state estimates $(q_a, \lambda_a, \hat{\pi}_{a,m})$,
- logs everything for evaluation.

3.2 Confidence to Switch (When Is $P(m | c)$ Trustworthy?)

We switch to adaptive control only when:

- coverage is sufficient: most vehicles occupy cells with $N_c \geq N_{\min}$,
- uncertainty is low: average entropy is below a threshold:

$$H(c) = - \sum_{m \in \{L, T, R\}} P(m | c) \log P(m | c),$$

- perception is healthy: detection/tracking FPS and ID stability meet minimum requirements.

Practical switching rule Switch if, over the last 5 minutes:

- $\geq 80\%$ of tracked vehicles are in “supported” cells ($N_c \geq N_{\min}$), and
- average entropy over supported cells $\leq H_{\max}$.

3.3 Adaptive Control Mode (Online Control + Continue Learning)

After switching, the controller uses real-time state estimates to choose signal actions. Importantly:

- **the system continues learning** and keeps updating $P(m | c)$ online,
- if perception quality degrades, the system can fall back to fixed-time or actuated mode,
- logs remain on to support evaluation and debugging.

3.4 Baseline Adaptive Controller (Student-Friendly)

We recommend starting with a **video-actuated** controller (simple) and a **max-pressure** controller (strong baseline).

- Video-actuated: extend green when queue is nonzero; end on gap/empty.

- Max-pressure: choose phase that serves the largest imbalance between upstream congestion and downstream availability (if downstream ROIs exist); otherwise use upstream queues only as an approximation.

3.5 Safety Supervisor (Always On)

Regardless of control method:

- enforce minimum/maximum green,
- apply yellow and all-red intervals,
- enforce pedestrian constraints if present,
- prevent phase-flipping via hysteresis / dwell time.

4 Algorithms (Implementation Pseudocode)

4.1 Online Learning of $P(m \mid c)$ from Exit Zones

Algorithm 1 Online Update of Spatial Intention Prior $P(m \mid c)$

Require: Track trajectories with IDs; exit zones for $m \in \{L, T, R\}$; decay factor β ; support threshold $N_{\min}$

- 1: **for** each completed track k **do**
 - 2: Determine final movement label m_k by first exit zone entered
 - 3: **for** each time step t in track history within learning region **do**
 - 4: $c \leftarrow \text{cell}(x_k(t), y_k(t))$
 - 5: $N_{c, m_k} \leftarrow (1 - \beta)N_{c, m_k} + \beta$
 - 6: $N_c \leftarrow (1 - \beta)N_c + \beta$
 - 7: **end for**
 - 8: **end for**
 - 9: Compute $P(m \mid c) = \frac{N_{c, m} + \alpha}{N_c + 3\alpha}$ for cells with $N_c \geq N_{\min}$
-

4.2 Switching Logic (Fixed-Time $\rightarrow$ Adaptive)

Algorithm 2 Shadow Mode and Safe Switching to Adaptive Control

Require: thresholds: $N_{\min}$, coverage $\rho_{\min}$, entropy limit $H_{\max}$

```
1: Start in shadow mode while intersection runs fixed-time
2: while system is running do
3:   Detect + track vehicles; update  $P(m \mid c)$ 
4:   Estimate  $q_a(t)$ ,  $\lambda_a(t)$ ,  $\hat{\pi}_{a,m}(t)$ 
5:   Compute cell support coverage  $\rho$  and mean entropy  $\bar{H}$ 
6:   if  $\rho \geq \rho_{\min}$  and  $\bar{H} \leq H_{\max}$  then
7:     Enable adaptive control (keep learning online)
8:   else
9:     Remain in shadow mode (fixed-time or actuated fallback)
10:  end if
11: end while
```

5 Working Plan (From Simulation Videos to Field Deployment)

We will develop in three stages: **(S1) simulation videos**, **(S2) real videos**, **(S3) field pilot**. Each stage has concrete deliverables.

5.1 Stage S1: Simulated Videos (Fast Iteration)

Objectives

- Validate the full pipeline end-to-end with controlled conditions.
- Generate repeatable experiments for comparisons (fixed-time vs actuated vs adaptive).
- Tune cell size, learning region, and switching criteria.

Tasks

1. Build a simulation intersection and generate videos (or replayable rendering).
2. Implement detection + tracking on the simulated videos.
3. Define approach ROIs and exit zones in the video frame.
4. Implement online learning of $P(m \mid c)$ and visualize heatmaps.
5. Implement state estimation: q_a , λ_a , $\hat{\pi}_{a,m}$.
6. Implement baseline controllers and the switching logic.
7. Run evaluation across multiple demand scenarios and report KPIs.

Deliverables

- Demo video with overlay (IDs + trajectories + exit zone labels).

- Heatmap visualization of $P(L | c)$, $P(T | c)$, $P(R | c)$.
- Baseline comparison plots: delay/queue/throughput.
- A short “lessons learned” memo: cell size, learning region, switching thresholds.

5.2 Stage S2: Real Intersection Videos (Robustness)

Objectives

- Validate robustness under real lighting, occlusion, and camera artifacts.
- Confirm that the lane-free intention prior still converges.
- Measure tracking stability and counting accuracy with spot-checks.

Tasks

1. Acquire real video clips (day + night if possible).
2. Calibrate minimal geometry (optional): stabilize camera, define ROIs/zones.
3. Fine-tune the detector if needed (small labeled dataset).
4. Re-run the full pipeline in shadow mode and validate:
 - turning counts vs manual samples,
 - queue estimates vs manual samples,
 - stability of $P(m | c)$ over time.
5. Stress-test the switching rule (do not actuate signals yet; shadow mode only).

Deliverables

- Real-video overlay demo (at least 2 conditions).
- Error report: turning count error, queue estimation error (spot checks).
- Updated $P(m | c)$ heatmaps and convergence curves.

5.3 Stage S3: Field Pilot (Last Step)

Objectives

- Integrate with signal controller interface (or run in recommendation/shadow mode first).
- Demonstrate safe transition and safe fallback.
- Provide DOT-ready documentation and safety checklist.

Tasks

1. Hardware setup (edge device + camera feed + network).
2. Real-time performance benchmark (FPS, end-to-end latency).
3. Shadow mode evaluation: system recommends signal actions but does not actuate.
4. If approved: limited adaptive control window with strict safety supervision.

5. Prepare final deployment report and funding proposal materials.

Deliverables

- Deployment checklist (power/network/storage/security).
- Safety supervisor validation report.
- Pilot results summary (before/after KPIs, with confidence intervals if possible).
- Proposal-ready documents: cost estimate, scalability plan, risk mitigation.

6 Milestone Schedule (12 Weeks Student Plan)

This plan assumes one student (adjustable).

Week	Focus	Primary Deliverables
1	Repo + video ingestion + basic YOLO inference	Ingestion script + overlay demo on sample clips
2–3	Tracking (ByteTrack) + trajectory logging	Stable IDs + trajectory logs + debugging notes
4	Define ROIs (approaches) + exit zones + self-labeling	Correct movement labeling on completed tracks
5	Implement $P(m c)$ + heatmap visualization + smoothing/decay	Heatmaps + convergence plots + chosen cell size
6	State estimation ($q_a, \lambda_a, \hat{\pi}_{a,m}$)	Time-series dashboard + manual spot-check report
7	Implement fixed-time replay baseline + video-actuated baseline	Baseline controller runs in simulation
8	Implement switching logic + confidence metrics	Shadow-to-adaptive demo in simulation
9	Implement max-pressure baseline (if feasible) + compare	Comparative plots (delay/queue/throughput)
10	Robustness tests (multiple demand seeds; noise)	Final simulation report + ablations
11	Apply to real video clips (shadow mode only)	Real-video overlay + updated error report
12	Final packaging: reproducible code + report + proposal material	Final report + demo video + deployment checklist

7 What to Log (Non-Negotiable)

Students must log:

- detection threshold, model version, FPS and latency,
- tracking ID switches (approx.) and track fragmentation counts,
- zone-crossing events (entry line, stop line, exit zones),

- N_c support counts and entropy statistics for switching,
- controller actions (phase, duration) and computed state $(q_a, \lambda_a, \hat{\pi})$,
- periodic snapshot frames with overlays for debugging.

8 Success Criteria

- Pipeline runs end-to-end on simulation videos in real time or near real time.
- Self-labeling via exit zones produces reliable movement classes.
- $P(m | c)$ converges (clear spatial patterns) and adapts online.
- Adaptive control improves at least one KPI (delay/queue/throughput) versus fixed-time in simulation.
- System includes safe switching + safe fallback and continues learning after switching.

9 Proposal/Funding Alignment (DOT-Friendly Talking Points)

This project is naturally appealing for DOT funding because:

- camera-only solution reduces infrastructure cost versus loop detectors,
- lane-free learning reduces site engineering effort,
- shadow mode enables safe evaluation before actuation,
- modular design supports future scaling and upgrades,
- logs and KPIs support accountability and auditing.